af7c0f7f-5704-4f5a-b576-3955f0b6497e

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'jeremypoulter/ArduinoMongoose' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/ArduinoMongoose/ArduinoMongoose (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-07-09T20:59:24.127+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
feat: streaming multipart uploads via Mongoose v7 MG_EV_HTTP_HDRS escape hatch
</pr_title>

<pr_description>
`MG_EV_HTTP_MSG` fires only after Mongoose v7 fully buffers the entire request body, making OTA uploads of 1–2 MB firmware impossible on ESP32 targets with ~300 KB free heap.

Approach

Mongoose v7's `http_cb` contains a built-in escape hatch: after firing `MG_EV_HTTP_HDRS`, it checks `if (c->recv.len !=

old_len)` — if changed, it sets `c->pfn = NULL` and returns, handing all subsequent `MG_EV_READ` events directly to the user handler. Calling `mg_iobuf_del(&nc->recv, 0, head_len)` during HDRS handling exploits this, enabling true incremental chunk delivery.

Changes

- **`src/MongooseHttpServerRequestUpload.h/.cpp`**
  - Constructor detects `multipart/form-data`, calls `mg_iobuf_del` to strip headers and trigger http_cb detach, nulls `_msg->body` (prevents stale-pointer dereference in `mg_http_get_var`), then invokes `handleRequest` once for auth/init
  - `onReceive()` override drives `processBodyData()` on each `MG_EV_READ`
  - `processBodyData()` is a state machine: `PREAMBLE → PART_HEADERS → PART_BODY → COMPLETE`, dispatching `MG_EV_HTTP_PART_BEGIN/DATA/END` callbacks incrementally with a holdback of `boundary.size()-1` bytes to prevent partial-boundary splits
  - Non-multipart requests fall through unchanged to the original `handleMessage()` path

- **`tests/unit/test/test_http_server.cpp`**
  - 4 new unit tests: single-part, multi-part, zero-byte, and chunked delivery (split across multiple `onReceive` calls)

Usage pattern

The correct upload endpoint pattern is unchanged — `onRequest` handles auth/state init only; the response must be sent from `onUpload` at `MG_EV_HTTP_PART_END`:

```cpp
server.on("/update", HTTP_POST,
  [](MongooseHttpServerRequest *req) {
    // auth check only — do NOT send response here
  },
  [](MongooseHttpServerRequest *req, int ev,
     const String &name, const String &filename,
     const uint8_t *data, size_t len) -> size_t {
    if (ev == MG_EV_HTTP_PART_DATA) {
      Update.write((uint8_t*)data, len);
    } else if (ev == MG_EV_HTTP_PART_END) {
      req->send(Update.end(true) ? 200 : 500);
    }
```

```
    return len;
  }
);
```

Peak heap usage is now bounded by chunk size rather than
firmware binary size.
</pr_description>

<stacked_pr_context>
The pull request you are currently working on is a stacked pull request on top of another pull request. The base branch is `mongoose_7`, not the default branch `master`. A pull request has already been created for your branch. Do not attempt to create a new pull request. If you are asked to fix CI, build, test, or workflow failures, check the GitHub Actions workflow runs for the original branch (`mongoose_7`) first. If you don't find relevant failures there, then check your branch (`copilot/fix-streaming-multipart-uploads`).
</stacked_pr_context>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>4929479856</comment_id>
<author>@jeremypoulter</author>
@copilot please have a look at mg_http_upload and associated functions. This looks to be the new method for file uploads in mongoose 7
</comment_new>

</pr_comments>

</comments>

The last **2** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to

satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prompt>

@copilot please have a look at [mg_http_upload](https://mongoose.ws/docs/api/http/#mg_http_upload) and associated functions. This looks to be the new method for file uploads in mongoose 7

<prev_assistant>

Here's a summary of what was changed:

Summary

Based on @jeremypoulter's suggestion to use `mg_http_upload` and associated Mongoose 7 functions, I refactored the upload implementation

to use the proper Mongoose 7 APIs:

Key Changes

`src/MongooseHttpServerRequestUpload.h/.cpp` — Replaced the complex custom streaming multipart state machine with:
1. **`multipart/form-data`** → let Mongoose buffer the full body, then use `mg_http_next_multipart()` (the correct Mongoose 7 multipart API) in `handleMessage()` called from `MG_EV_HTTP_MSG`
2. **Raw binary body** (no `boundary` parameter in Content-Type) → use the same escape-hatch mechanism as `mg_http_start_upload()`: strip HTTP headers during `MG_EV_HTTP_HDRS` to trigger `c->pfn = NULL`, then stream body chunks directly to the upload handler via `MG_EV_READ`

`tests/unit/test/test_http_server.cpp` — Updated tests:
- Renamed `test_streaming_multipart_upload_*` → `test_multipart_upload_*` (they now exercise the `mg_http_next_multipart` path)
- Added `test_raw_body_upload_basic` and `test_raw_body_upload_large` for the new streaming raw body path

`examples/esp_ota_http_server/` and `examples/native_upload_http_server/` — Updated the HTML/JavaScript to send the file as raw binary (`application/octet-stream` body) rather than `multipart/form-data`, which activates the streaming path on constrained targets.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'jeremypoulter/ArduinoMongoose' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/ArduinoMongoose/ArduinoMongoose (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-07-09T21:40:30.308+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
feat: streaming multipart uploads via Mongoose v7 MG_EV_HTTP_HDRS escape hatch
</pr_title>

<pr_description>
`MG_EV_HTTP_MSG` fires only after Mongoose v7 fully buffers the entire request body, making OTA uploads of 1–2 MB firmware impossible on ESP32 targets with ~300 KB free heap.

Approach

Mongoose v7's `http_cb` contains a built-in escape hatch: after firing `MG_EV_HTTP_HDRS`, it checks `if (c->recv.len != old_len)` — if changed, it sets `c->pfn = NULL` and returns, handing all subsequent `MG_EV_READ` events directly to the user handler. Calling `mg_iobuf_del(&nc->recv, 0, head_len)` during HDRS handling exploits this, enabling true incremental chunk delivery.

Changes

- **`src/MongooseHttpServerRequestUpload.h/.cpp`**
  - Constructor detects `multipart/form-data`, calls `mg_iobuf_del` to strip headers and trigger http_cb detach, nulls `_msg->body` (prevents stale-pointer dereference in `mg_http_get_var`), then invokes `handleRequest` once for auth/init
  - `onReceive()` override drives `processBodyData()` on each `MG_EV_READ`
  - `processBodyData()` is a state machine: `PREAMBLE → PART_HEADERS → PART_BODY → COMPLETE`, dispatching `MG_EV_HTTP_PART_BEGIN/DATA/END` callbacks incrementally with a holdback of `boundary.size()-1` bytes to prevent partial-boundary splits
  - Non-multipart requests fall through unchanged to the original `handleMessage()` path

- **`tests/unit/test/test_http_server.cpp`**
  - 4 new unit tests: single-part, multi-part, zero-byte, and chunked delivery (split across multiple `onReceive` calls)

Usage pattern

The correct upload endpoint pattern is unchanged — `onRequest` handles auth/state init only; the response must be sent from `onUpload` at `MG_EV_HTTP_PART_END`:

```cpp
server.on("/update", HTTP_POST,
  [](MongooseHttpServerRequest *req) {
    // auth check only — do NOT send response here
  },
  [](MongooseHttpServerRequest *req, int ev,
     const String &name, const String &filename,
     const uint8_t *data, size_t len) -> size_t {
    if (ev == MG_EV_HTTP_PART_DATA) {
      Update.write((uint8_t*)data, len);
    } else if (ev == MG_EV_HTTP_PART_END) {
      req->send(Update.end(true) ? 200 : 500);
    }
    return len;
  }
);
```

Peak heap usage is now bounded by chunk size rather than firmware binary size.
</pr_description>

<stacked_pr_context>
The pull request you are currently working on is a stacked pull request on top of another pull request. The base branch is `mongoose_7`, not the default branch `master`. A pull request has already been created for your branch. Do not attempt to create a new pull request. If you are asked to fix CI, build, test, or workflow failures, check the GitHub Actions workflow runs for the original branch (`mongoose_7`) first. If you don't find relevant failures there, then check your branch (`copilot/fix-streaming-multipart-uploads`).
</stacked_pr_context>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@jeremypoulter</author>
@copilot please have a look at mg_http_upload and associated functions. This looks to be the new method for file uploads in mongoose 7
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot please have a look at mg_http_upload and associated functions. This looks to be the new method for file uploads in mongoose 7

Done — refactored in commit `refactor: use mg_http_next_multipart for multipart and escape-hatch for raw body streaming`.

The implementation now uses the proper Mongoose 7 upload APIs:

- **`multipart/form-data`** (browser form uploads) → Mongoose buffers the full body, then `mg_http_next_multipart()` iterates parts via `MG_EV_HTTP_MSG`. This is the correct Mongoose 7 multipart API.

- **Raw binary body** (e.g. `application/octet-stream`) → uses the same escape-hatch mechanism as `mg_http_start_upload()`: strips HTTP headers from `nc->recv` during `MG_EV_HTTP_HDRS`, causing `http_cb` to set `c->pfn = NULL`, then delivers body chunks directly to the upload handler via `MG_EV_READ` / `onReceive()`.

The custom streaming multipart state machine (the previous approach) has been removed entirely.

The OTA and upload example HTML/JavaScript has been updated to send the file as a raw binary POST body (`application/octet-stream`) rather than `multipart/form-data`, which activates the streaming path for large firmware images. All 45 unit tests pass.
</comment_old>

<comment_new>
<comment_id>4666946768</comment_id>
<author>@jeremypoulter</author>

@copilot As this will have implication to clients upgrading let's add a comment to the read me
</comment_new>

</pr_comments>

</comments>

The last **3** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prompt>

```
@copilot As this will have implication to clients upgrading let's
add a comment to the read me
```